AIORI 3 HACKATHON | SMART CITIES | PS-003

Federated Traffic-Flow Prediction

Problem understanding, proposed solution, optimization plan, architecture, data flow evaluation, and 15-day implementation plan

Team Query
Rohit Kumar Das and Sneha Saha
Faculty Mentor: Sayani Manna

ABSTRACT

Objective

- **Train a shared traffic-flow forecasting model across 3–5 simulated intersections.**
- **Keep raw local training records on each client.**
- **Measure the communication cost of collaborative training.**

Method

- Use a public traffic dataset and chronological split.
- Train a compact GRU on 3–5 virtual clients using FedAvg.

Experiments

- Compare centralized and federated training.
- Test IID and non-IID data splits across 3 random seeds.

Evaluation

- Prediction error: MAE and RMSE.
- Convergence and upload/download bytes per round.
- Total communication bytes across rounds.

Expected contribution: a reproducible simulator and an evidence-based accuracy–communication trade-off analysis.

PROBLEM DESCRIPTION

Traffic sensors produce observations over time. A city wants to predict future traffic flow, but continuously sending every raw observation to a central cloud can create bandwidth and data-governance concerns.

Traffic prediction

Use recent sensor observations to estimate future vehicle flow.

Federated learning

Train locally on each simulated client and send model updates for server aggregation.

Main research question

How do non-IID traffic patterns and communication overhead affect prediction quality and convergence?

Official scope: simulator only; no live cameras, city-wide deployment, or formal privacy guarantees are required.

BACKGROUND

Traffic-flow forecasting

A time-series task: use past flow values to estimate a future value, such as the next 5-minute interval.

Federated Averaging

Clients train the same model locally. The server averages their model parameters, weighted by local sample counts.

Non-IID data

Clients may have different traffic levels, rush-hour peaks, and variability. This can make collaboration harder.

Key distinction

1. Federated Learning avoids routinely uploading raw training records in this simulation; model updates are still transmitted.
2. Federated Learning alone does not guarantee that updates reveal no information.
3. Communication cost must be measured alongside model error.

LITERATURE & RFC REVIEW

McMahan et al. (2017)

Introduced the FedAvg approach for communication-efficient learning from decentralized data. Core algorithm reference named in the official brief.

DCRNN / STGCN

Traffic forecasting research illustrates temporal patterns and road-network relationships. Useful future extensions; not required for the first MVP.

Flower framework

A practical framework for orchestrating federated clients and a server. Optional: a small custom FedAvg simulator may be easier to debug first.

Resources

FedAvg paper: https://proceedings.mlr.press/v54/mcmahan17a.html

DCRNN code: https://github.com/liyaguang/DCRNN

STGCN paper: https://www.ijcai.org/Proceedings/2018/0505

Flower documentation: https://flower.ai/docs/framework/index.html

PROPOSED SOLUTION

Dataset

Use one public traffic dataset with time-indexed sensor flow. Candidate: a PeMS-derived traffic dataset such as PEMS04, subject to verifying availability, license, and column definitions.

Forecasting model

Start with a compact GRU. Input: recent traffic observations; output: next-step flow prediction. Compare against a simple persistence/mean baseline.

Federated simulator

Create 3–5 virtual clients, train locally, aggregate with FedAvg, evaluate the global model on a held-out chronological test period.

Planned experiment matrix

- Centralized vs federated training under comparable data splits.
- IID vs non-IID client partitioning; report mean and variability over 3 random seeds.
- Report MAE/RMSE, convergence across rounds, and communication bytes per round.

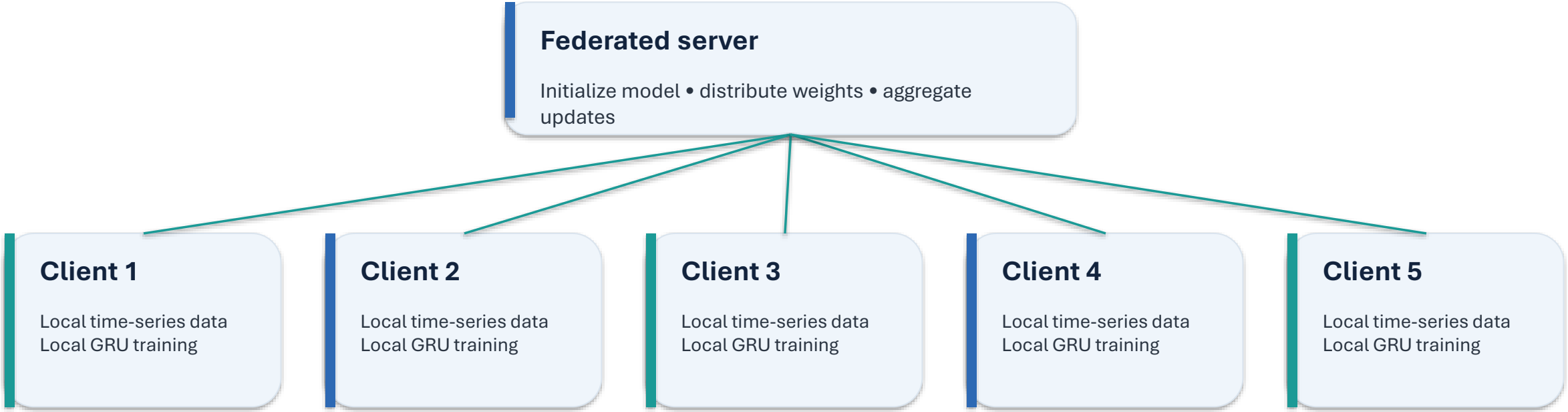
SOLUTION OPTIMIZATION

Primary question: Can communication be reduced without an unacceptable increase in prediction error?

Scenario	What changes	Measure
Baseline	FedAvg with standard floating-point parameters	MAE/RMSE; bytes per round
Reduced communication	Fewer communication rounds or fewer local-update exchanges	Cumulative bytes; convergence
Optional compression	Lower-precision update serialization, if time permits	Payload bytes; MAE/RMSE

Do not claim a saving before measurement. Report the exact byte-accounting convention and separate estimated model payload from actual serialized bytes.

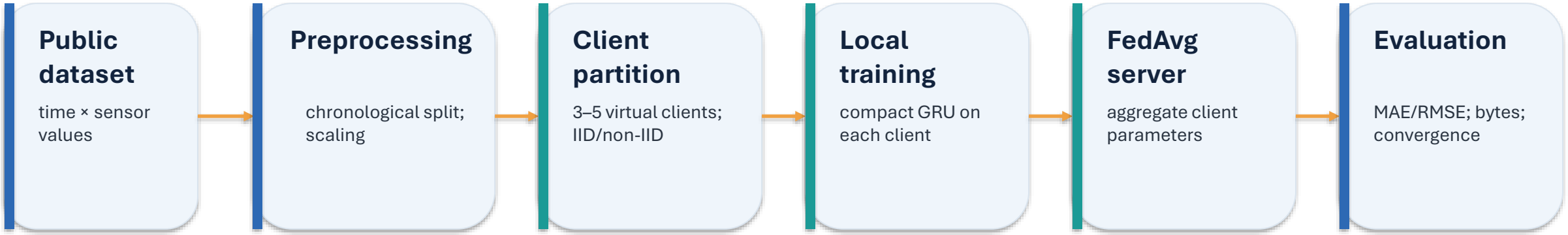
SOLUTION ARCHITECTURE & DESIGN



Server sends global parameters → clients train locally → clients return updates → server applies FedAvg → repeat for R rounds.

Evaluation module: global model metrics + per-round communication log + experiment configuration and random seed.

DATA FLOW DIAGRAM (DFD)



Federated round: server → clients (global weights) → local training → server (model updates) → FedAvg aggregation.

Guardrails

- Fit scaling/preprocessing on training data only; preserve chronological order.
- Keep a consistent held-out test interval and document client-partition logic.
- Record per-round upload/download bytes and the random seed for reproducibility.

DEPLOYMENT TIMELINE

Days 1–2	✓ Understand brief; complete and upload structured proposal PDF; create private GitHub repository and add required collaborator.
Days 3–4	✓ Set up Python environment; inspect dataset; build preprocessing and persistence baseline.
Days 5–7	✓ Train compact GRU; validate chronological split and MAE/RMSE.
Days 8–10	✓ Implement 3–5 clients and FedAvg; compare centralized/federated and IID/non-IID.
Days 11–12	✓ Run 3 seeds; log bytes per round and convergence; check calculations.
Days 13–14	✓ Try one communication optimization if baseline works; prepare plots/report.
Day 15	✓ Reproduce results; rehearse pseudocode demo; finalize repository and report.

BIBLIOGRAPHY

McMahan, B. et al. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. AISTATS. https://proceedings.mlr.press/v54/mcmahan17a.html

Li, Y. et al. (2018). Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. ICLR. Code: https://github.com/liyaguang/DCRNN

Yu, B., Yin, H., and Zhu, Z. (2018). Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting. IJCAI. https://www.ijcai.org/Proceedings/2018/0505

Flower Federated Learning Framework documentation. https://flower.ai/docs/framework/index.html

LibCity traffic prediction toolkit and datasets. https://github.com/LibCity/Bigscity-LibCity

AIORI 3 Hackathon. PS-003: Federated Traffic-Flow Prediction with Communication-Cost Evaluation. Official problem statement provided by organizers.

Reporting integrity

All performance values, communication savings, and convergence outcomes will be reported only after experiments are run. No results are claimed in this proposal.

Thank You

Questions & feedback

PS-003: Federated Traffic-Flow Prediction with Communication-Cost Evaluation

AIORI 3 Hackathon | Team: Query